GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

PenguinHarness：面向1000+模型的高效自进化Agent Harness

PenguinHarness: Efficient Self-Evolving Agent Harness for 1000+ Models



郑耀威 Yaowei Zheng

千影光流联合创始人兼 CTO，PenguinHarness、LlamaFactory作者

Co-founder & CTO, PrismShadow；PenguinHarness & LlamaFactory Creator

GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

PenguinHarness：一站式 Agent 自进化平台

郑耀威　LlamaFactory作者、 千影光流 CTO



01

RSI 平台解放 Agent 框架的束缚



通用 Agent 很强，但生产环境仍需定制 Agent

Claude Code

Codex

通用能力强、任务覆盖广

01

适应垂域任务

业务知识、流程与工具高度专用

02

降低部署成本

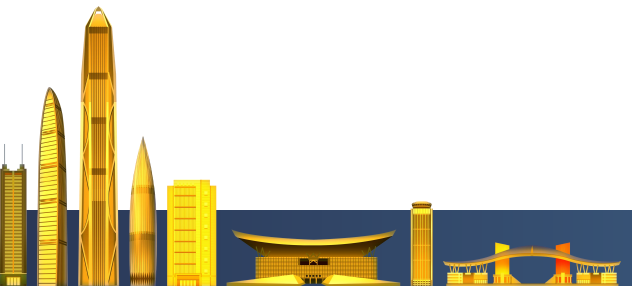
Claude/GPT 账单成本高昂

03

数据自主安全

企业隐私数据不能泄露

真正进入生产的，不是一个“万能 Agent”，而是一组可控、可部署、可迭代的定制 Agent



通用 Agent 在遇到行业问题水土不服

- **知识鸿沟：**知道行业术语，不等于掌握行业知识
- **评价鸿沟：**语言正确，不等于业务正确
- **流程鸿沟：**行业工作不是一道孤立的题目
- **异常鸿沟：**正常流程容易，边缘情况困难
- **系统鸿沟：**会使用工具，不等于能操作企业系统
- **责任鸿沟：**能做决定，不等于有权做决定

$$P(\text{全部成功}) = 0.95^{10} \approx 59.9\%$$

通用智能体，单步成功率看起来已经很高，整条流程的成功率却不及格

现有Agent框架无法原生自进化，调优费时费力



LangChain

Trace、成本、评估

LangGraph

思考、分支、循环

LangSmith

LLM、RAG、工具

LangFuse

监控运行

01 高学习成本

先理解框架，才能表达需求

02 长交付周期

每个应用都要重新搭建

03 低效率复用

经验停留在人的脑海

02

PenguinHarness 核心 RSI 能力



全球首个支持自进化的开源智能体框架

上一代

人工手动调优大语言模型

人 → 零代码 Web UI → 训练 / 改进模型

- LlamaFactory

70K+ GitHub Stars

3M+ 累计下载

500+ 支持模型

下一代

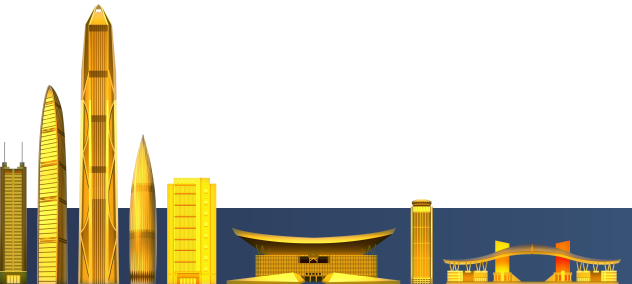
让 Agent 自动改进 Agent

AI → SDK+文件系统 → 构建 / 改进 Agent

- PenguinHarness

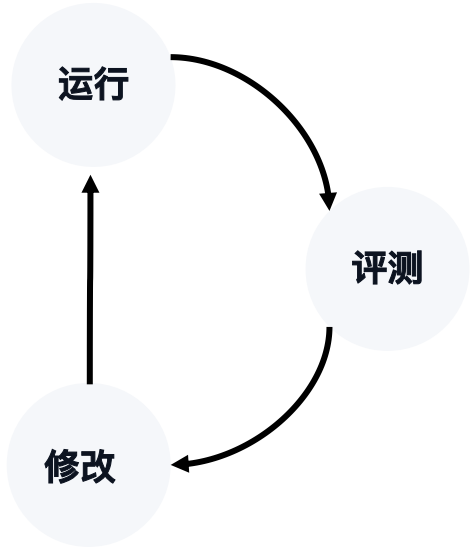
已于 2026 年 7 月 正式发布

GitHub 突破 2600 Stars



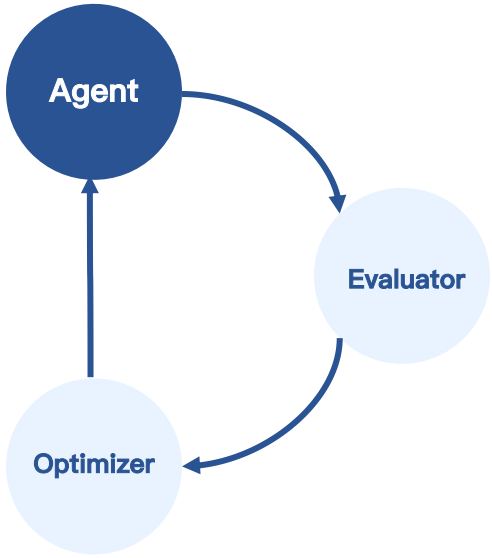
PenguinHarness 让 AI 进化迈向 2.0 全自动

传统调优



Human in the loop

Self-Evolving



Agents in the loop

自进化必须要有边界

Harness

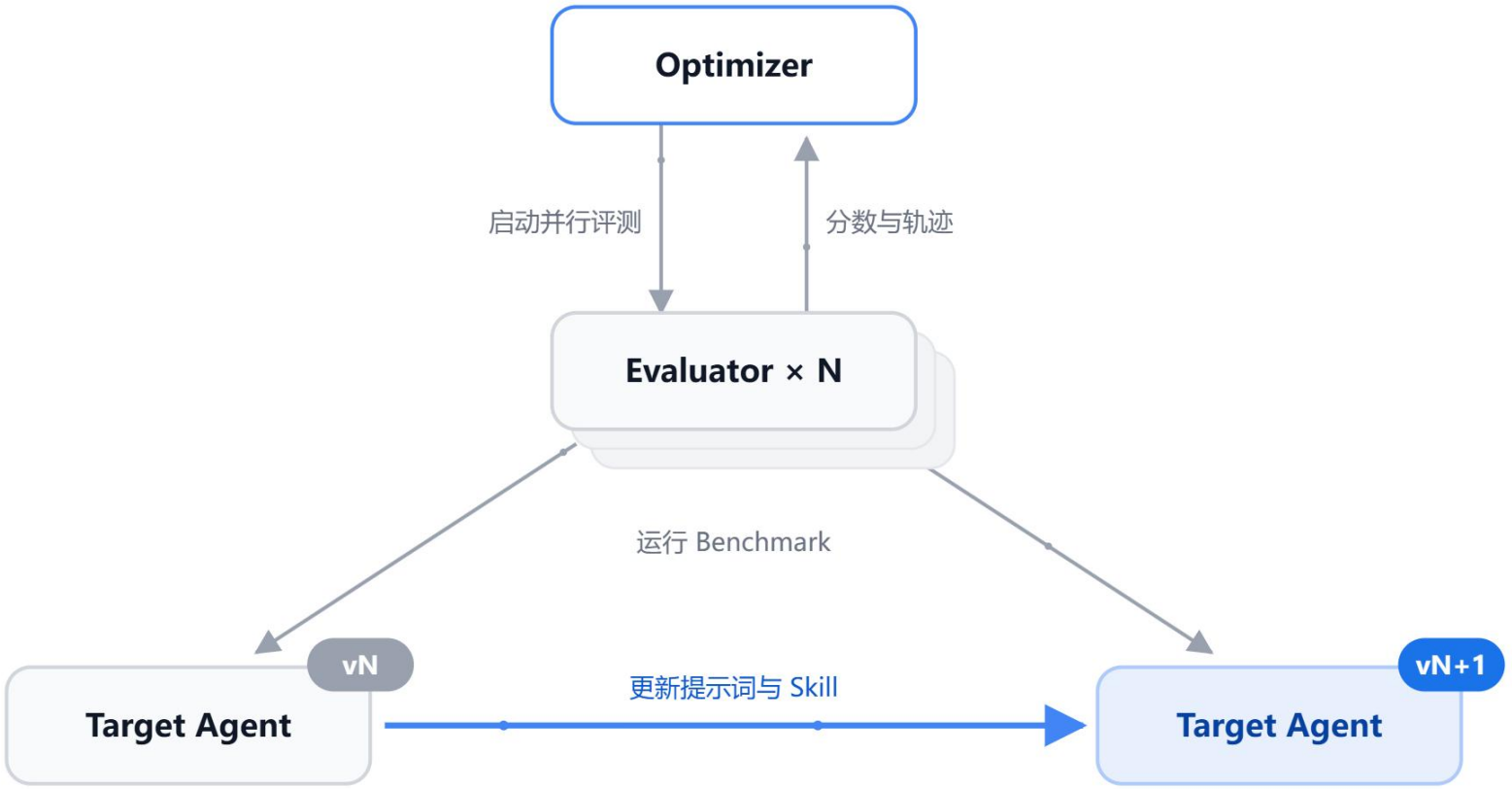
固定执行规则

评测隔离

权限与回滚

Making agents change is easy. Making them improve is hard.

不同角色职责的 Agents 会组成一个自进化闭环



- 组织评测
- 独立打分
- 分析优化
- 验证迭代

抽丝剥茧，将 Agent 自进化的成本降到最低



Penguin Message

Simplest Is the Best

坚持最小化工具集与简洁的底层接口，以更少的工具调用与 Token 消耗，高效完成复杂任务。

Penguin SDK

Harness for Building Agents

通过 PenguinHarness SDK，让 Agent 从零自主完成 Agent 应用的构建。

Penguin Skills

Harness for Recursive Self-Improvement

通过 PenguinHarness Skills，Agent 以自我评估与自我优化实现递归式自我提升。

Single Source of Truth

文件系统是唯一真相来源

Prompt 不硬编码

状态不藏在黑盒

SDK 接口保持干净

最小化 Harness 负担，二次开发友好

Claude Code 工具集

read_file	write_file	list_dir
patch	grep	find
browser	memory	planner
shell	fetch	search

工具越多，Schema 越长

误调用与上下文成本越高

PenguinHarness 工具集

$ shell

一个通用接口，承载绝大多数工具操作

6	72	On-demand
内置工具	行系统提示词	Skill 按需加载

更少工具 · 更少 Token · 更少犯错

防止野蛮进化，内置“契约”：约束自进化边界

01

Rubric 答案隔离

被优化 Agent 只能看到题面，不能泄露答案

02

限定修改范围

只允许改 Workspace 与 Skills，Harness 框架固定

CONTRACT.md

我们为自进化定制Harness

03

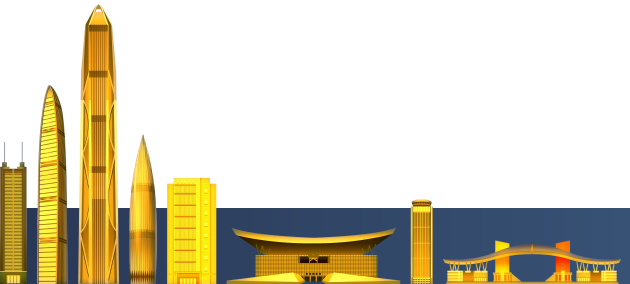
全流程文件记录

模型请求、工具调用、审批决定统一写入 Trace 文件

04

快照与回滚

候选版本未严格提升，自动恢复本轮修改



03

使用RSI框架可以做哪些高价值赋能?



GOSIM

GOSIM Shenzhen 2026

用途一

无需专业 FDE 也能落地企业 Agent



一句话 + ¥0.2，Agent 自动造出 Agent

提示词

收集 Claude Code 文档，做一个回答带来源引用的配置专家

- ✓ 收集文档
- ✓ 构建检索
- ✓ 生成界面
- ✓ 验证引用

¥0.2 PenguinHarness + DeepSeek V4 Pro 流式回答成本

¥43.8 Codex + GPT模型 回答混乱成本



PenguinHarness 已落地成功案例



体检报告质控

一家体检机构使用 PenguinHarness 自动开发了报告质控 Agent，并将它部署到院内生产环境，接入本地 Qwen3 14B，数据不出机房。过去一轮人工复核要 30 分钟，现在每分钟自动核查 30 份报告，结论与医学专家基本一致，审核产能提升数倍。



产线设备巡检

一家制造企业使用 PenguinHarness 自动开发了产线巡检 Agent，并将它部署到多条流水线的生产环境。Agent 全天候监测设备状态，发现异常后优先尝试自动恢复；上线后停机时间减少 65%，产出提升约 2 倍。

进化覆盖 Agent 开发的全生命周期

围绕进化覆盖 Agent 开发全生命周期的核心目标，让 RSI 发生在 Agent 应用生命周期的每一个节点，从建立、评估、测试、反馈到轨迹的每一步都能精准覆盖，为 Agent 的持续迭代优化筑牢基础。



实现多端覆盖， 多消息渠道接入



用途二

使主流 Agent 拥有自进化能力

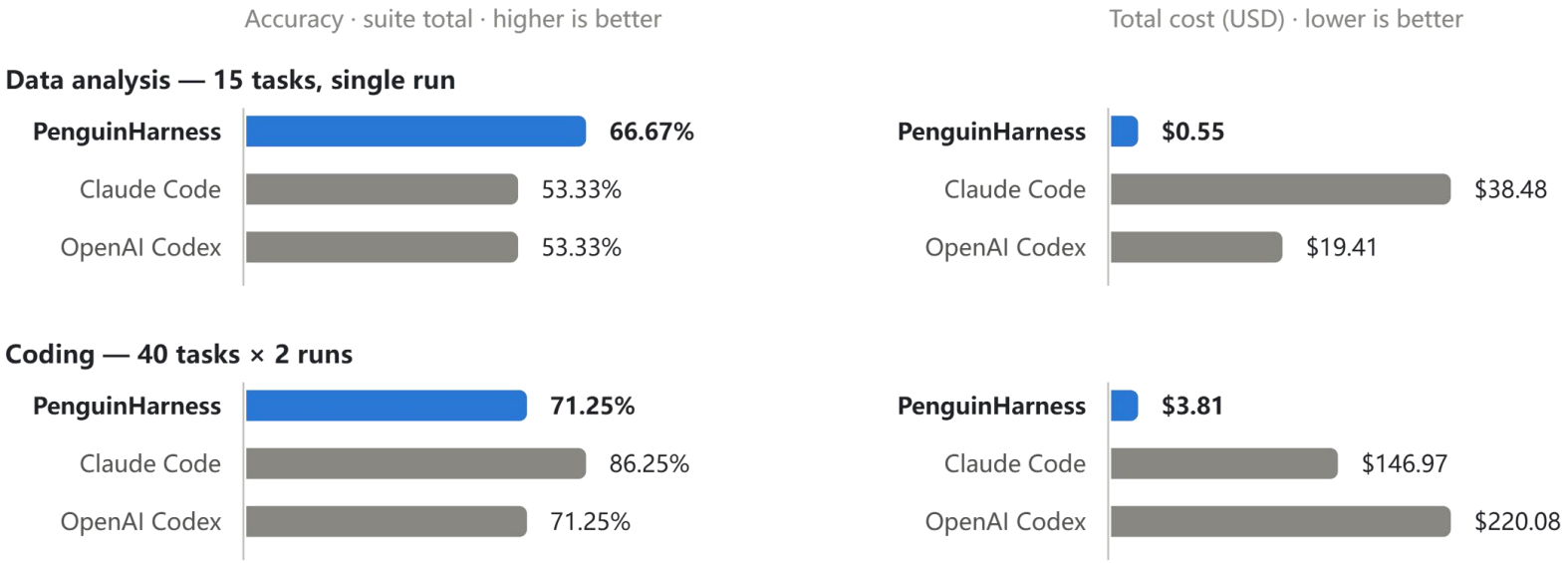


让主流智能体拥有自进化能力



- 全兼容协议，无需额外适配即可快速支持主流智能体；
- 不改变原有智能体使用习惯，让进化潜移默化的发生着；
- 根据不同 Agent harness 的特点，提供专属的进化方式、评估方式。

用 1/70 的成本，做到 Codex 同等甚至更好的准确率



Each harness runs the model it is normally paired with: PenguinHarness on DeepSeek V4 Pro, Claude Code on Claude Opus 4.8, OpenAI Codex on GPT-5.5. Accuracy, Tokens and cost are suite totals at official pricing.

1 / 70

成本
vs. Claude/Codex

66.67%

数据分析准确率最高

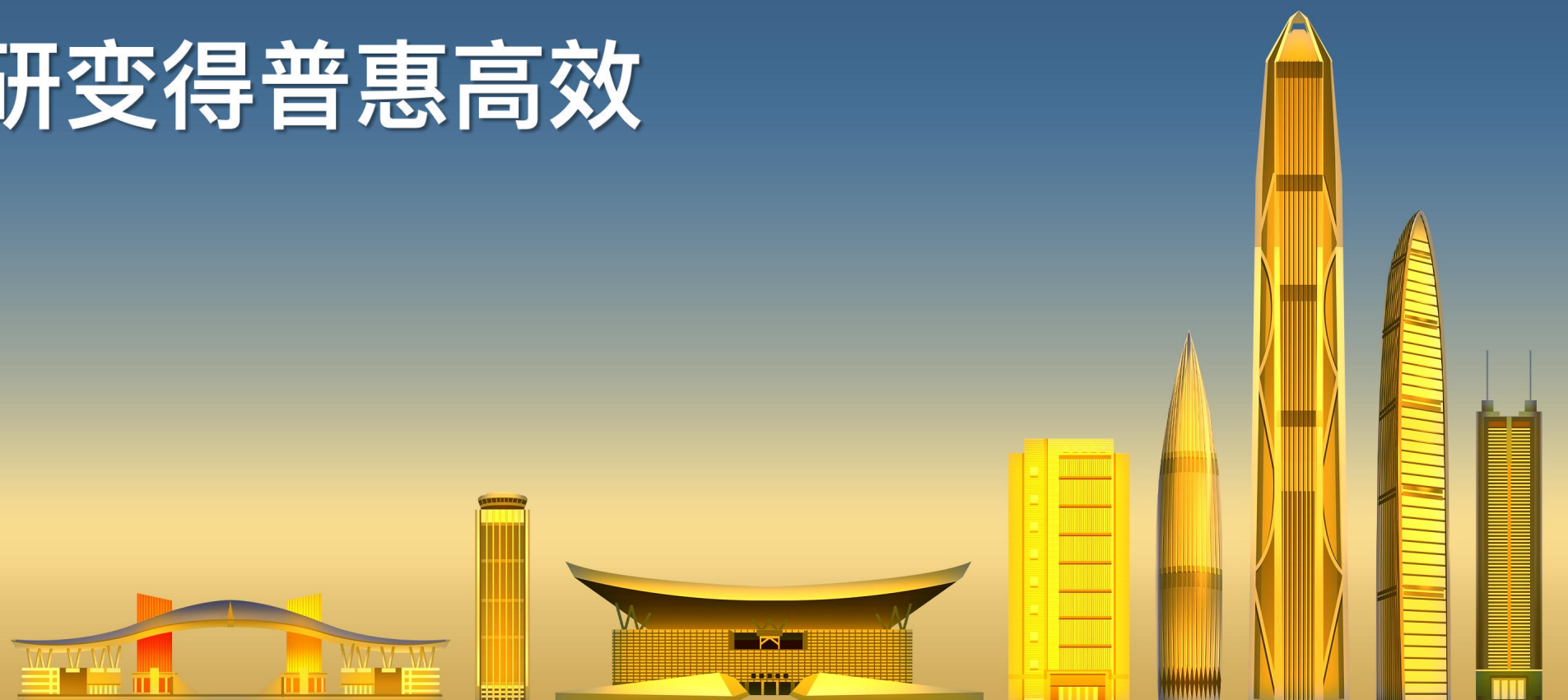
71.25%

代码任务与 Codex 持平

同一批任务，每个 Harness 搭配其常用模型；成本为整套任务总成本。

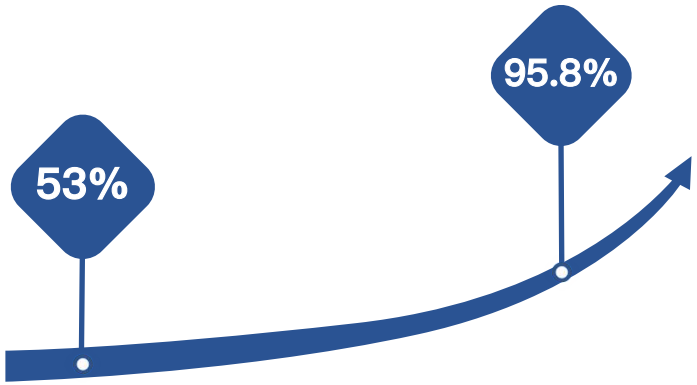
用途三

让 RSI 科研变得普惠高效



让每个人都能在本地跑通 Agent 自进化闭环

借助技能库，Agent 自己评估、自己优化：Optimizer 组织多个 Evaluator 并行打分，依据分数与运行轨迹定位失分原因，把 Agent 从版本 N 优化到版本 N+1



花费：约13分钟 + 0.52元



科学进化：先造尺子，再改 Agent

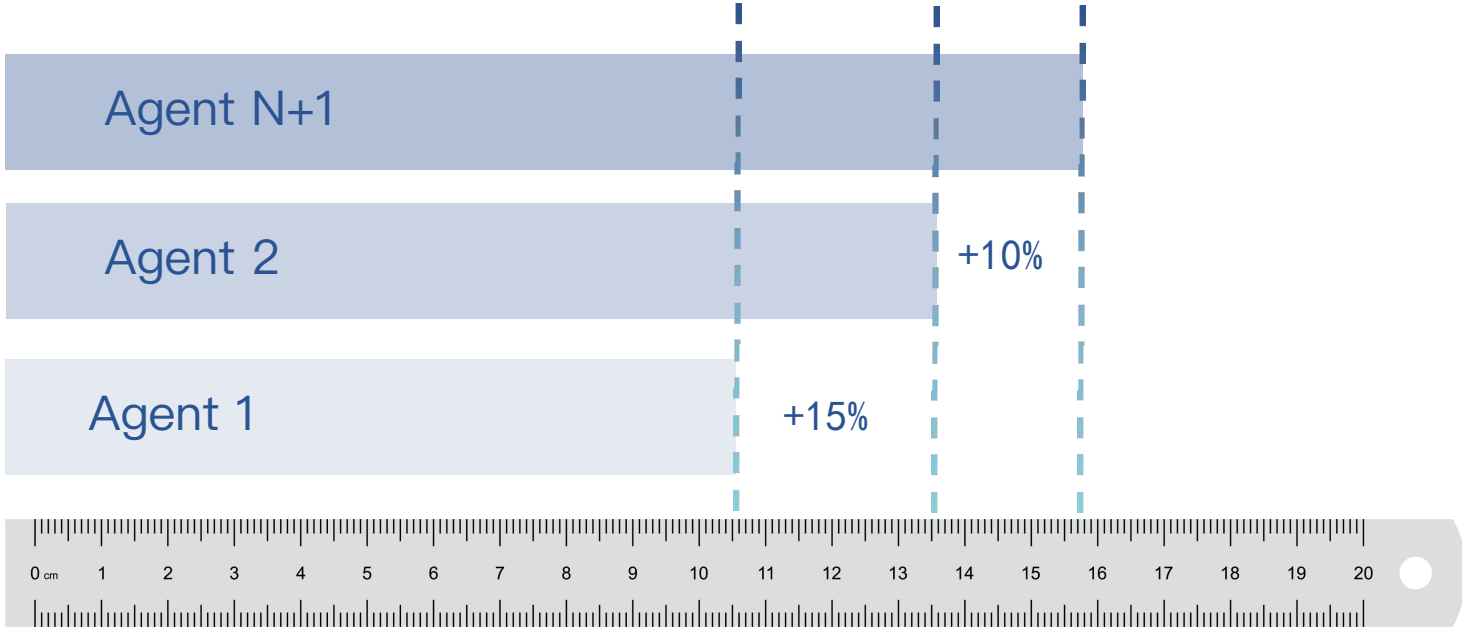
在提示词即源代码，无评审，无diff 的情况下，保证改造结果正确 —— 创建评估器；

Penguinharness遵循评估优先原则，通过评估器创建技能快速帮助用户打造可衡量结果的尺子。

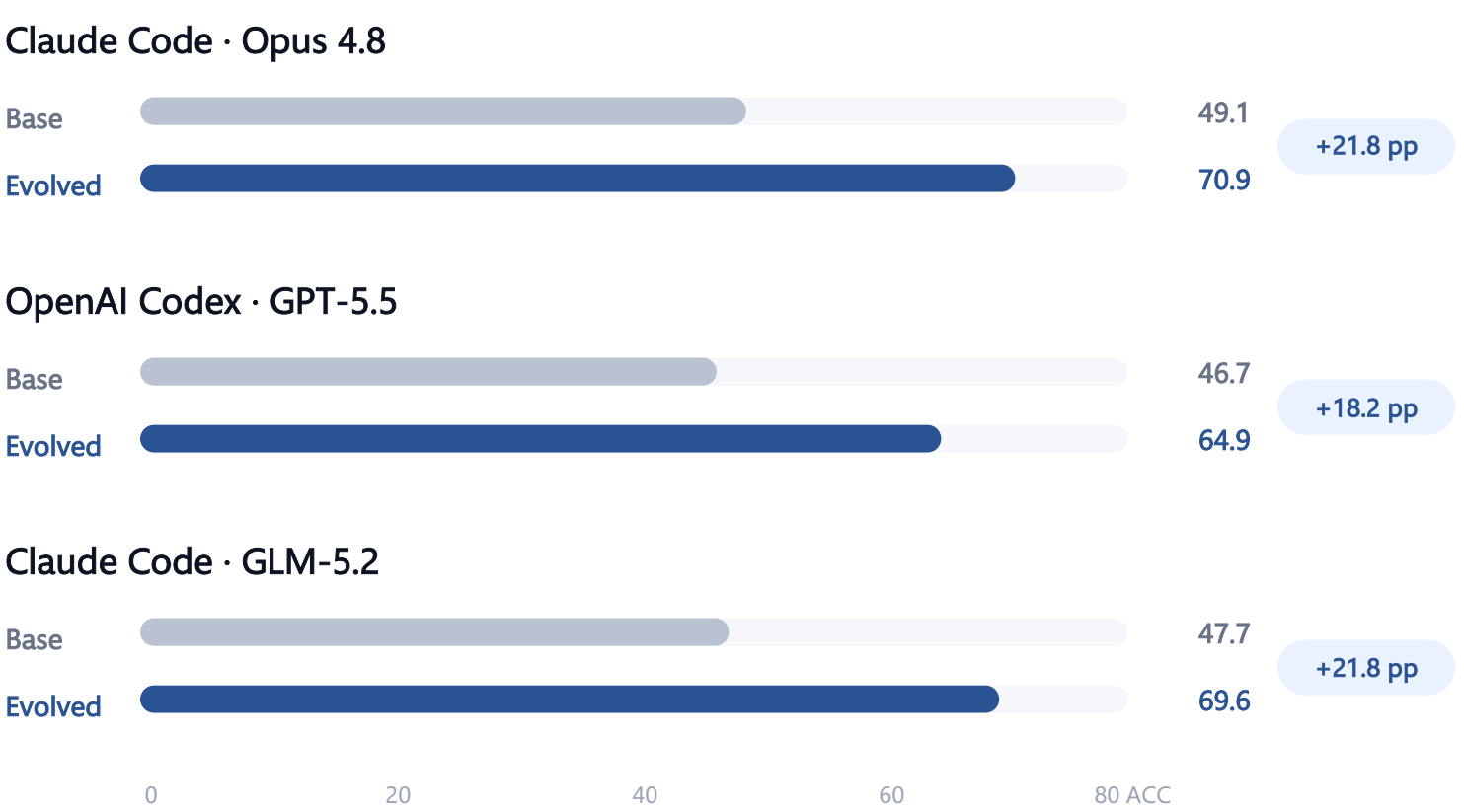
尺子

Benchmark Design

创建 Case：每个 Case 包含公开的 statement/ 和私有的 rubric/；Rubric 满分固定 100 分，含可观察评分项和部分给分。



我们用 GDPevo 测出 Agent 自进化上限



+18 → +22 pp 自进化能让 GLM 超过 Claude

GDPevo

GDPevo: Evaluating Agent Self-Evolution on Real Business Tasks

Leijun Zhou[1], Zhihao Liu[1], Xiang Qu[1], Chenxu Liu[1], Yifei Liu[1], Yanke Yu[1], Jingzhe Xu[1], Xuejun Wu[1], Buyue Qian[1], Xi Chen[2], Yaowei Zheng[1], Junhao Hu[1]

[1]PrismShadow, [2]New York University

Agent self-evolution updates an agent's persistent state from prior experience and reuses it to solve related tasks more effectively. Evaluating self-evolution is difficult: existing benchmarks provide limited coverage of economically valuable task domains; do not always design training and test tasks such that test-time gains can be attributed to training experience; and remain vulnerable to data contamination. We present *GDPevo*, an evolution-native benchmark grounded in GDP-related enterprise workflows, together with the fully automated data pipeline that generates it. Its core mechanism, **rule hybridization**, decomposes each enterprise workflow into atomic business rules, distributes subsets of these rules across training tasks, and recombines them in held-out test tasks, so that test-time gains are attributable. *GDPevo* spans CRM, ERP, finance, healthcare, legal, and data-centric workflows. Its V1 release contains 120 tasks in 12 groups, with 5 training and 5 held-out test tasks per group; full automation lets the pipeline expand the suite to 240 tasks in 24 groups (V2) within two days, providing a practical response to contamination. Using *GDPevo*, we evaluate four agents (harness + model) under four supervision types. Self-evolution consistently improves held-out accuracy by up to 16.44 pp. But the best evolved agents remain far below a fully informed oracle ceiling of 91.6%, indicating that the self-evolution ability of current agents remains far from fully realized. We publicly release the pipeline, benchmark, and full evaluation results: https://github.com/Prism-Shadow/GDPevo.

Date: August 5, 2026
Correspondence: Junhao Hu (hu@prismshadow.com), Yaowei Zheng (zheng@prismshadow.com)

arXiv:2608.03764v1 [cs.AI] 4 Aug 2026

1 Introduction

In the AI era, once a task can be *automated* and *evaluated*, it is rarely far from being solved at scale. To bring these two capabilities to agentic task solving, agent self-evolution has emerged as a promising paradigm. Under this paradigm, an agent continuously improves task performance by updating its internal state, such as parametric states and non-parametric states (e.g., memory, skills, prompts, or harness code), based on prior interactions and reusing the resulting state in subsequent related tasks [1]. Automation includes techniques, such as continual learning [2], experiential learning [3, 4], and recursive self-improvement [5, 6], for performing these internal updates. Evaluation, on the other hand, determines whether these updates actually improve subsequent behavior. Although the literature now offers many techniques to automate self-evolution, evaluation methodology remains less mature.

We focus on evolution benchmarks that measure learning through an explicit train–test split[1], falling into two categories. First, evolution-native benchmarks are designed to test whether an agent learns from training experience and transfers what it learns to held-out tasks. They deliberately construct the train–test split so that the capabilities an agent acquires on the training tasks are genuinely required and exercised on the test tasks [8, 9, 10, 11]. Second, evolution-adaptive benchmarks repurpose task suites originally designed for static, single-episode capability measurement, such as interactive reasoning environments and software-engineering tasks [3, 4, 12, 13]. They introduce no deliberate design and directly split the suite into training and test sets, reporting whether the experience gained while solving the training set improves performance on the test set.

[1]A separate family of benchmarks instead measures evolution over time, tracking how performance improves with accumulated experience within a single long-horizon deployment (e.g., EdgeBench [7]) rather than across a train–test split.

GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

